

DAA Assignment - 1

Name: Asneem Athar Shaik

Date: 06-03-2025

Slot: E₂ + TE₂

Reg. No: 22 BCE 8807

1. How can we use the greedy approach with Find and Union [Disjoint Set] to schedule jobs such as that the total profit is maximized while ensuring jobs meet their deadlines?

Eg: Consider the following jobs represented as

[Job ID, Deadline, Profit]

Jobs = $\{(J_1, 2, 100), (J_2, 1, 50), (J_3, 2, 10)\}$

Using Find and Union, allocate the latest available slot for each job for maximizing profit.

Approach:

- Sort jobs by profit in descending order.
- Use Disjoint Set [Find & Union] to track available slots.
- Allocate each job to the latest available slot before its deadline.

Example: Given Jobs

Jobs = $\{(J_1, 2, 100), (J_2, 1, 50), (J_3, 2, 10)\}$

Step-1:

Sort jobs by profit [Descending order]:

$(J_1, 2, 100), (J_2, 1, 50), (J_3, 2, 10)$

Step-2:

Find maximum deadline

$\max(\text{Deadlines}) = 2$

Create an array to track available slots using

Disjoint Set ;

Slots = $[-1, -1, -1]$ (0-based index, size = max deadline + 1)

Step-3: process the job, One by One, using Find & Union.
We try to assign each job to the latest available slot,
before or on its deadline.

Job J_1 (Deadline = 2, Profit = 100)

- * Use Find(2) to check the latest available slot ≤ 2 .
- * Slot 2 is free, so assign J_1 to slot 2.
- * Union (Find(2), Find(1)) $\rightarrow$ Merge slot 2 with slot 1.

Slots = $[-1, -1, J_1]$

Job J_2 (Deadline = 1, profit = 50)

- * Use Find(1) to check the latest available slot ≤ 1 .
- * Slot 1 is free, so assign J_2 to slot 1.
- * Union (Find(1), Find(0)) $\rightarrow$ Merge slot 1 with slot 0.

Slots = $[-1, J_2, J_1]$

Job J_3 (Deadline = 2, profit = 10)

- * Use Find(2) $\rightarrow$ Slot 2 occupied
- * check Find(1) $\rightarrow$ Slot 1 occupied
- * No available slot, so J_3 is not scheduled.

Compute total profit:

$$= 100 + 50 = 150$$

3. How does Rabin-Karp algorithm efficiently search for a pattern in a given text using hashing? Explain how rolling hash helps in reducing redundant computations.

Eg: Give the text "ABCCDABCD" and the pattern "BCD", use the Rabin-Karp algorithm with a simple hash function (sum of ASCII values) to find occurrence of pattern in a text.

How it works:

1. Compute the hash of the pattern P.

2. Compute the hash of the first substring of the text T [of the same length as P]

3. Slide the window over the text and update the hash efficiently using a rolling hash.

4. If the hash matches, compare the actual strings to verify the match.

Rolling Hash Concept:

* Time Complexity $O(n+m)$ on average.

* Instead of recomputing the hash from scratch for each new substring, the rolling hash efficiently update the hash by:

• Removing the contribution of leftmost character.

• Adding the contribution of new character.

character.

Eg: Finding "BCD" in "ABCCDABCD"

Text (T) : "ABCCDABCD"

Pattern (P) : "BCD"

Hash Function: Sum of ASCII values of characters

Step-1: Compute Hash of Pattern 'BCD'

$$H(P) = H("BCD") = 66 + 67 + 68 = 201$$

Step-2: Compute Initial Hash of First Substring of length 3 in Text.

$$H(T_0) = H("ABC") = 65 + 66 + 67 = 198$$

Step-3: Slide the window using rolling hash

Each step remove the ASCII of the left most character and add the ASCII of the new right most character.

1. Window 1 ("ABC"):

$$H = 65 + 66 + 67 = 198 \rightarrow \text{No match}$$

2. Window 2 ("BCC"):

$$H = (198 - 65 + 67) = 200 \rightarrow \text{No match}$$

3. Window 3 ("CCD"):

$$H = (200 - 66 + 68) = 202 \rightarrow \text{No match}$$

4. Window 4 ("CDA"):

$$H = (202 - 67 + 65) = 200 \rightarrow \text{No match}$$

5. Window 5 ("DAB"):

$$H = (200 - 68) + 66 = 198$$

6. Window 6 ("ABC"):

$$H = (198 - 65) + 67 = 200$$

7. Window 7 ("BCD"):

$$H = (200 - 66) + 68 = 201$$

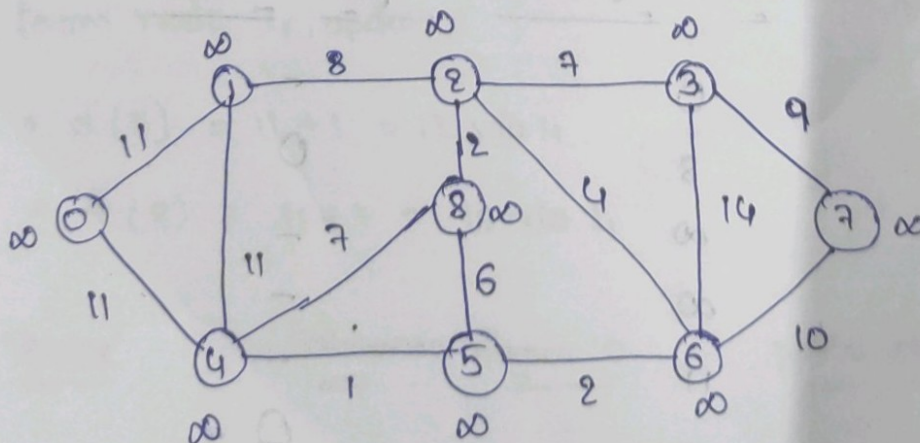
→ Match!

Since hash matches, we compare "BCD" with the pattern "BCD" and confirm the occurrence at index 6.

Pattern "BCD" found at index 6.

→ By using rolling hash, the algorithm efficiently updates the hash in constant time instead of recomputing it from scratch, making it much faster than a naive search.

Apply Dijkstra's algorithm to find the shortest path from 0 to all other vertices, compute this mathematically with tables.



Step-1% Initialization

We maintain a table to track the shortest known distance to each vertex and update it iteratively.

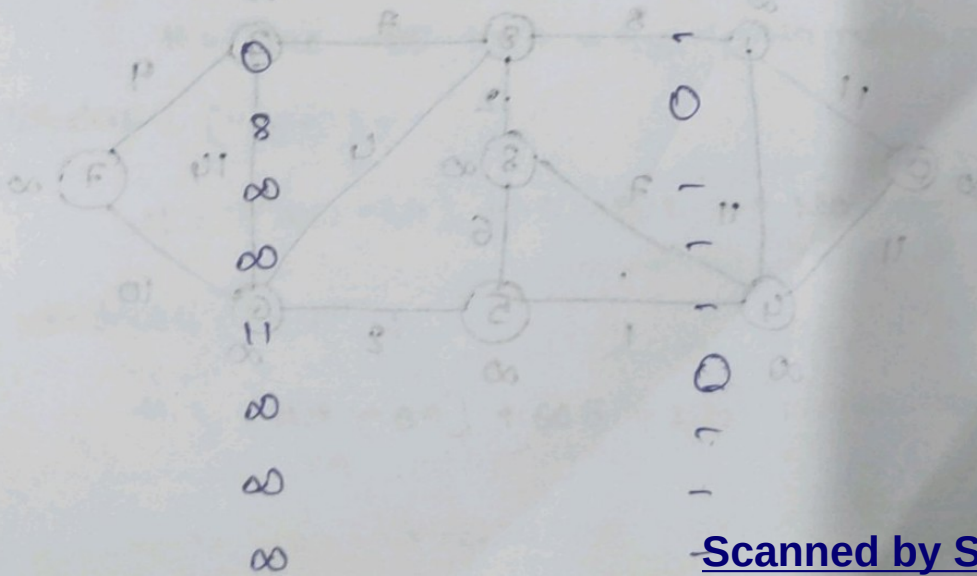
<u>Vertex</u>	<u>Distance from 0</u>	<u>parent Vertex</u>
0	0	
1	∞	
2	∞	
3	∞	
4	∞	
5	∞	
6	∞	
7	∞	
8	∞	

$$d(1) = 8 \text{ via } 0$$

$$d(4) = 11 \text{ via } 0$$

update distances for directly connected nodes from 0

<u>Vertex</u>	<u>Distance from 0</u>	<u>parent Vertex</u>
0	0	
1	8	0
2	∞	
3	∞	
4	11	0
5	∞	
6	∞	
7	∞	



<u>Vertex</u>	<u>Distance from 0</u>	<u>parent vertex</u>
0	0	-
1	8	-

Step-2: Visit Node 1 [Minimum Distance 8]

from node 1, update distances;

$$d(2) = 8 + 8 = 16 \text{ via } 1$$

<u>Vertex</u>	<u>Distance from 0</u>	<u>parent vertex</u>
0	0	-
1	8	-
2	16	1
3	∞	-
4	11	-
5	∞	-
6	∞	-
7	∞	-
8	∞	-

Step-3: Visit Node 4 [Minimum Distance 11]

from node 4, update;

$$+ d(5) = 11 + 1 = 12 \text{ via } 4$$

$$+ d(8) = 11 + 7 = 18 \text{ via } 4$$

<u>Vertex</u>	<u>Distance from 0</u>	<u>parent vertex</u>
0	0	-
1	8	-

<u>Vertex</u>	<u>Distance from 0</u>	<u>parent vertex</u>
0	0	-
1	8	0
2	16	1
3	∞	-
4	11	0
5	12	4
6	∞	-
7	∞	-
8	18	4

Step-4%

Visit Node 5 [Minimum Distance 12]

From 5, update;

$$d(6) = 12 + 2 = 14 \text{ via } 5$$

<u>Vertex</u>	<u>Distance from 0</u>	<u>parent vertex</u>
0	0	-
1	8	0
2	16	1
3	∞	-
4	11	0

⑧

Vertex

Distance from 0

previous vertex

5

12

4

6

14

7

∞

5

8

18

-

4

Step 5:

Visit Node [Minimum Distance 14]

from 6, update:

$$d(7) = 14 + 10 = 24 \text{ via 6}$$

Vertex

Distance from 0

previous vertex

0

0

-

1

8

0

2

16

1

3

∞

-

4

11

0

5

12

4

6

14

5

7

24

from 6

8

18

4

Step-6:

Visit Node 2 [Minimum Distance 16]

from 2, update:

- $d(3) = 16 + 7 = 23$ via 2

- $d(8) = 16 + 2 = 18$ (already 18)

<u>vertex</u>	<u>Distance from 0</u>	<u>parent vertex</u>
0	0	
1	8	0
2	16	1
3	23	2
4	11	0
5	12	4
6	14	5
7	24	6
8	18	4

Step-7: visit Node 8 [Minimum Distance 18]

from 8, update;

- $d(3) = 18 + 4 = 22$ via 8 (shorter than 23)

<u>vertex</u>	<u>Distance from 0</u>	<u>parent vertex</u>
---------------	------------------------	----------------------

vertex

Distance from 0

parent vertex

1

8

0

2

16

1

3

22

8

4

11

0

5

12

6

14

7

24

8

18

6

4

Step-8 : Visit Node 3 [Min Distance 22]

from 3, update :

• $d[7] = 22 + 9 = 24$ [same, no update]

vertex

Distance from 0

parent vertex

0

0

1

8

0

2

16

1

3

22

8

4

11

0

5

12

4

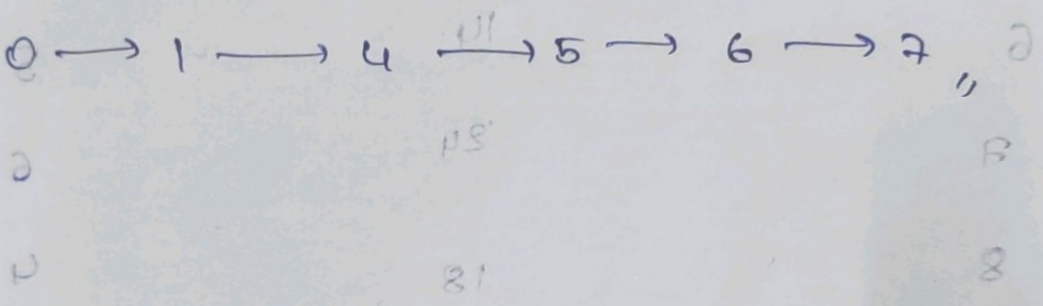
Distance from 0

vertex

parent vertex

<u>vertex</u>	<u>parent vertex</u>	<u>Distance from 0</u>
0		0
6	1	14
7	2	24
8	0	18

Final shortest paths :



[99 shortest path] & total distance : 8+9+8

Distance from 0 : 8+9+8 = 25

vertex

Distance from 0

parent vertex

<u>vertex</u>	<u>parent vertex</u>	<u>Distance from 0</u>
0		0
1	0	8
2	1	14
3	2	24
4	0	18

Assignment -2

Name: Asneem Athar Shah

Date: 14-03-2025

Regd. NO: 22BCE8807

Slot: E2+TE2

1. 0/1 Knapsack Problem

You are given a knapsack with a weight capacity of 15 kg. There are 7 items with the following properties:

Item	Weight (kg)	Value (\$)
------	-------------	------------

1	2	3
2	5	10
3	10	20
4	5	8
5	1	1
6	7	15
7	3	7

Task: Find the maximum value you can obtain while ensuring that the total weight does not exceed 15 kg.

Ans

Initialize $dp[0] = 0$ and compute for weight capacity 1 to 15.

Knapsack Capacity = 15 kg

Items list = (weight, value)

(2,3), (5,10), (10,20), (5,8), (1,1), (7,15), (3,7)

→ Define $dp[w]$ as the maximum value achievable with weight capacity w .

→ Base case: $dp[0] = 0$

Formula :

$dp[w] = \max(dp[w], dp[w - \text{weight of item}] + \text{value of item})$.

Initial DP table $dp = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]$

processing item 1 [weight = 2, value = 3] :

update $dp[w]$ for $w = 15$ down to 2

$$dp[w] = \max(dp[w], dp[w-2] + 3)$$

$$dp = [0, 0, 3, 3, \dots]$$

processing item 2 [w = 5, v = 10] :

$$dp[w] = \max(dp[w], dp[w-5] + 10)$$

$$dp = [0, 0, 3, 3, 3, 10, 10, 13, 13, 13, 13, 13, 13, 13, 13]$$

processing item 3 [w = 10, v = 20] :

$$dp[w] = \max(dp[w], dp[w-10] + 20)$$

$$dp = [0, 0, 3, 3, 3, 10, 10, 13, 13, 13, 20, 20, 23, 23, 23, 23, 30]$$

processing item 4 [w = 5, v = 8] :

$$dp[w] = \max(dp[w], dp[w-5] + 8)$$

$$dp = [0, 0, 3, 3, 3, 10, 10, 13, 13, 13, 20, 20, 23, 23, 23, 30]$$

processing item 5 [w = 1, v = 1] :

$$dp[w] = \max(dp[w], dp[w-1] + 1)$$

$$dp = [0, 1, 3, 4, 4, 10, 11, 13, 14, 14, 20, 21, 23, 24, 24, 24, 30]$$

Processing item 6 ($w=7, v=15$):

$$dp[w] = \max [dp[w], dp[w-7] + 15]$$

$$dp = [0, 1, 3, 4, 4, 10, 11, 15, 16, 18, 20, 25, 26, 28, 30, 31]$$

Processing item 7 ($w=3, v=7$):

$$dp[w] = \max [dp[w], dp[w-3] + 7]$$

$$dp = [0, 1, 3, 7, 8, 10, 11, 15, 16, 18, 20, 25, 26, 28, 30, 31]$$

The maximum value that can be obtained with 15 kg capacity is 32.

→ item 6 (7 kg, 15\$)

Total weight = 7+5+3 = 15 kg

→ item 3 (5 kg, 11\$)

Total value = 15+10+7 = 32\$

→ item 7 (3 kg, 7\$)

Profit	Weight	Items	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
-	-	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
3	2kg	1	0	0	3	3	3	3	3	3	3	3	3	3	3	3	3	3
10	5kg	2	0	0	3	3	3	10	10	13	13	13	13	13	13	13	13	13
20	10kg	3	0	0	3	3	3	10	10	13	13	13	20	20	23	23	23	30
8	5kg	4	0	0	3	3	3	10	10	13	13	20	20	23	23	23	23	30
1	1kg	5	0	1	3	4	4	10	11	13	14	14	20	21	23	24	24	30
15	7kg	6	0	1	3	4	4	10	11	15	16	18	20	21	23	24	24	30
7	3kg	7	0	1	3	7	8	10	11	15	16	18	23	23	25	26	26	32

2. Floyd - Warshall Algorithm:

A company has 4 offices, and the following adjacency matrix represents the weighted graph between them. Use Floyd - Warshall to compute the shortest paths between all pairs of nodes.

0	3	∞	7
8	0	2	∞
5	∞	0	1
2	∞	∞	0

Task: Compute the shortest paths between all pairs of nodes using Floyd - Warshall.

→ Given adjacency matrix:

	0	1	2	3
0	0	3	∞	7
1	8	0	2	∞
2	5	∞	0	1
3	2	∞	∞	0

* Here, ∞ [infinity represents no direct path between nodes]

* This algorithm updates iteratively the distance matrix using each node as an intermediate.

* The update rule: $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$.

→ For iteration $k=0$

	0	1	2	3
0	0	3	∞	7
1	8	0	2	15
2	5	8	0	1
3	2	5	∞	0

→ For iteration $k=1$

	0	1	2	3
0	0	3	5	7
1	8	0	2	15
2	5	8	0	1
3	2	5	7	0

→ For iteration $k=2$

	0	1	2	3
0	0	3	5	6
1	8	0	2	3
2	5	8	0	1
3	2	5	7	0

→ For iteration $k=3$

	0	1	2	3
0	0	3	5	6
1	7	0	2	3
2	3	6	0	1
3	2	5	7	0

* final shortest path matrix between all pairs of offices.

Algorithm:

All Paths (cost, A, n).

// cost $[1:n, 1:n]$ is the cost adjacency matrix of a graph with .

// n vertices; $A[i, j]$ is the cost of a shortest path from vertex .

// i to vertex j, cost $[i, i] = 0.0$, for $1 \leq i \leq n$.

{

for $i := 1$ to n do

for $j := 1$ to n do

$A[i, j] := \text{cost}[i, j]$; // copy cost into A.

for $k := 1$ to n do

for $i := 1$ to n do

for $j := 1$ to n do

$A[i, j] := \min(A[i, j], A[i, k] + A[k, j]);$

}

3. Travelling salesperson Problem [TSP]:

A company has 5 ware houses in different locations, The cost matrix is:

[0 20 30 10 11

15 0 16 4 2

3 5 0 2 4

19 6 18 0 3

16 4 7 16 0]

Task: Find the optimal route that minimizes the travel cost while visiting each ware house exactly once.

$$* g(1, V - \{1\}) = \min_{2 \leq k \leq n} \{c_{1k} + g(k, V - \{1, k\})\}$$

$$* g(i, S) = \min_{j \in S} \{c_{ij} + g(j, S - \{i\})\}$$

A B C D E

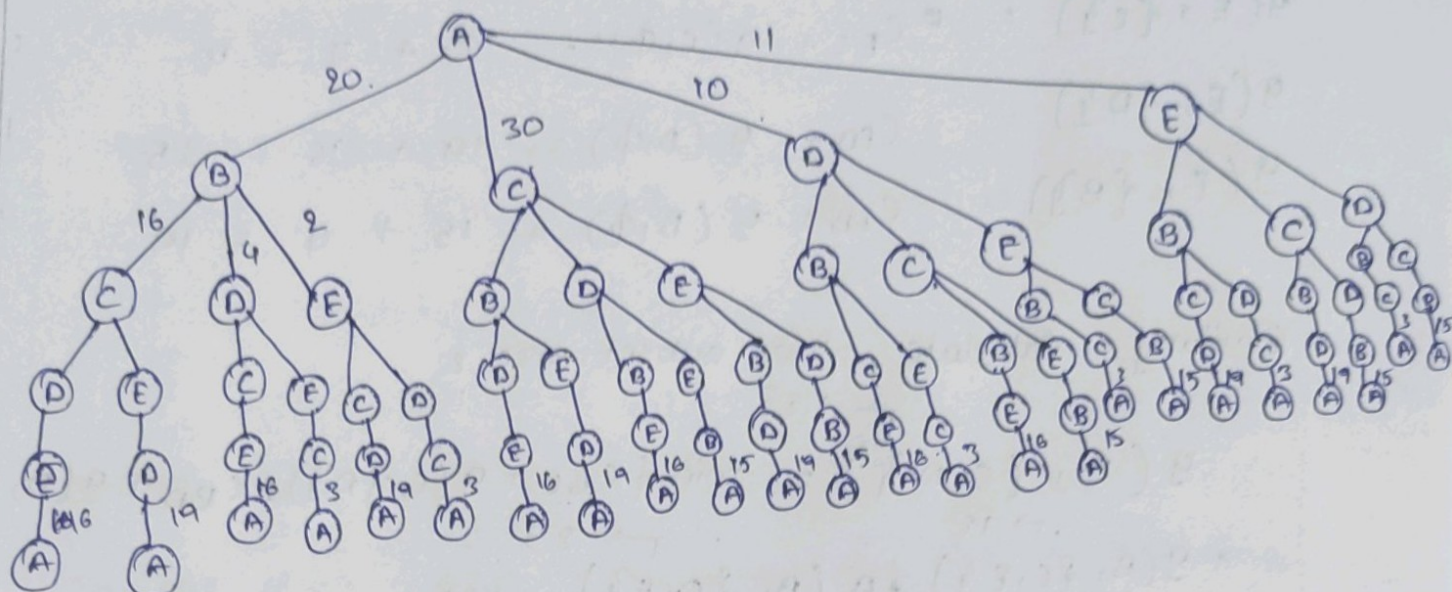
A 0 20 30 10 11

B 15 0 16 4 2

C 3 5 0 2 4

D 19 6 18 0 3

E 16 4 7 16 0



$$g(B, \phi) = 16$$

$$g(C, \phi) = 3$$

$$g(D, \phi) = 19$$

$$g(E, \phi) = 16$$

Ans; $g(i, s)$

Ans;

$$g(i, s) = \min_{j \in s} \{c_{ij} + g(j, s - \{j\})\}$$

$$g(B, \{C\}) = c_{BC} + g(C, \phi)$$

$$\Rightarrow 16 + 3 = 19$$

$$g(B, \{D\}) = c_{BD} + g(D, \phi)$$

$$= 4 + 19$$

$$= 23$$

$$g(B, \{E\}) = c_{BE} + g(E, \phi)$$

$$= 2 + 16 = 18$$

$$g(C, \{B\}) = c_{CB} + g(B, \phi) = 20$$

$$g(C, \{D\}) = c_{CD} + g(D, \phi) = 21$$

$$g(C, \{E\}) = c_{CE} + g(E, \phi) = 20$$

$$g(D, \{B\}) = c_{DB} + g(B, \phi) = 6 + 16 = 21$$

$$g(D, \{C\}) = c_{DC} + g(C, \phi) = 18 + 3 = 21$$

$$g(D, \{E\}) = c_{DE} + g(E, \phi) = 3 + 16 = 19$$

$$g(E, \{C\}) = c_{EC} + g(C, \phi) = 3 + 7 = 10$$

$$g(E, \{D\}) = c_{ED} + g(D, \phi) = 19 + 16 = 35$$

$$g(E, \{B\}) = c_{EB} + g(B, \phi) = 15 + 4 = 19$$

Similarly, Calculate for subset size 2

$$g(B, \{C, D\}) = \min \{c_{BC} + g(C, \{D\}), c_{BD} + g(D, \{C\})\} \rightarrow 25$$

$$* g(B, \{C, E\}) \rightarrow 12 \quad * g(B, \{D, E\}) \rightarrow 23$$

$$* g(E, \{B, D\}) \rightarrow 23 \quad * g(E, \{C, D\})$$

$$* g(D, \{B, C\}) \rightarrow 25 \quad * g(D, \{B, E\}) \rightarrow 13$$

$$* g(D, \{B, E\}) \rightarrow 22 \quad * g(C, \{B, E\}) \rightarrow 23$$

$$* g(E, \{B, D\}) \rightarrow 27 \quad * g(C, \{D, E\}) \rightarrow 21$$

$$* g(C, \{B, D\}) \rightarrow 23$$

for subset 3: Here $A \rightarrow 1, B \rightarrow 2, C \rightarrow 3, D \rightarrow 4, E \rightarrow 5$

$$g(2, \{3, 4, 5\}) = \min \{c_{23} + g(3, \{4, 5\}), c_{24} + g(4, \{3, 5\}),$$

$$c_{25} + g(5, \{3, 4\})\} = \min \{37, 19, 30\} = 19$$

Similarly;

$$g(3, \{2, 4, 5\}) = \min \{28, 24, 31\} = 24$$

$$g(4, \{2, 3, 5\}) = \min \{18, 41, 26\} = 18$$

$$g(5, \{2, 3, 4\}) = \min \{29, 30, 41\} = 29$$

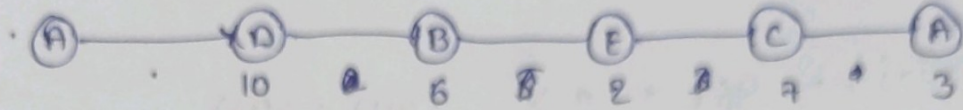
finally;

$$g(1, \{2, 3, 4, 5\}) = \min \{c_{12} + g(2, \{3, 4, 5\}) + c_{13} +$$

$$g(3, \{2, 4, 5\}) + c_{14} + g(4, \{2, 3, 5\}) + c_{15} + g(5, \{2, 3, 4\})$$

$$\Rightarrow \min \{37, 54, 28, 40\} = 28$$

route:



$$\Rightarrow 10 + 6 + 2 + 7 + 3 = 28 //$$

4) Optimal Binary Search Tree [OBST]

You are given 6 node, 7 external nodes with the following probabilities.

<u>Internal</u> <u>Nodes</u>	<u>Frequency</u>	<u>External node</u>	<u>Frequency</u>
5	10	0	18
10	15	1	5
15	25	2	16
20	30	3	12
25	20	4	8
30	5	5	5
		6	7

$$Q(i, j) = \min_{i \leq k \leq j} \{C(i, k-1) + C(k, j)\} + W(i, j)$$

$$W(i, j) = Q(i) + \sum_{l=i+1}^j (Q(l) + P(l))$$

Internal nodes $\rightarrow 5, 10, 15, 20, 25, 30$

External nodes $\rightarrow 18, 5, 16, 12, 8, 5, 7$

Internal node frequencies $\rightarrow 10, 15, 25, 30, 20, 5$

~~$W[0,0] = Q[0]$~~

Internal nodes

Base Case: $W[0,0] = Q[0] = 18$

$$W[1,1] = Q[1] + P[1] = 5 + 10 = 15$$

$$W[2,2] = Q[2] + P[2] = 16 + 15 = 31$$

$$W[3,3] = Q[3] + P[3] = 12 + 25 = 37$$

$$W[4,4] = Q[4] + P[4] = 8 + 30 = 38$$

$$W[5,5] = Q[5] + P[5] = 5 + 20 = 25$$

$$W[6,6] = Q[6] + P[6] = 7 + 5 = 12$$

Compute until $W[0,6]$

Calculate $C(i,j)$

$$C[0,0] = W[0,0] = 18$$

$$C[1,1] = W[1,1] = 15$$

$$C[4,4] = W[4,4] = 38$$

$$C[2,2] = W[2,2] = 31$$

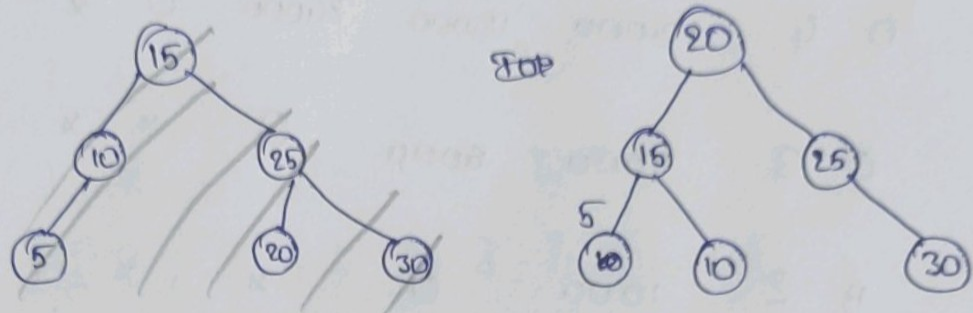
$$C[3,3] = W[3,3] = 37$$

$$C[5,5] = W[5,5] = 25$$

$$C[6,6] = W[6,6] = 12$$

Similarly compute until $L[0,6]$

Final Optimal Tree :



Total Cost :

142

Matrix chain multiplication;

you are given '5' matrices with dimensions

$$A = 5 \times 10$$

$$C = 20 \times 30$$

$$E = 40 \times 50$$

$$B = 10 \times 20$$

$$D = 30 \times 40$$

Task : find the optimal order to multiply them using matrix chain multiplication.

$$A = 5 \times 10$$

B

DP [] =

5

10

20

30

40

50

Sol:

6

	A	B	C	D	E
	1	2	3	4	5
E 5	20,000	38	64,000	60,000	0
D 4	10,000	18,000	24,000	0	x
C 3	4,000	6,000	0	x	x
B 2	1,000	0	x	x	x
A 1	0	x	x	x	x

Formula:

$$m[i, j] = \min_{i \leq k < j} \{ m[i, k] + m[k+1, j] + p_{i-1} p_k p_j \} \text{ if } i < j$$

$$c[i, j] = c[i, k] + c[k+1, j] + d[i-1] d[i] d[j]$$

$$= 0 + 0 + 5 \times 10 \times 20 = 1000$$

$$c[2, 3] = 6000$$

$$c[3, 4] = 24,000$$

$$c[4, 5] = 60,000$$

2 matrices

For 3 matrices at a time:

= = =

$C[1,3], C[2,4], C[3,5]$

↓ 4000 ↗ 18,000 ↘ 60,000

For 4 matrices at a time:

$C[1,4], C[2,5] \rightarrow 38,000$

↘ 10,000

For 5 matrices at a time:

$C[1,5] = 20,000$

∴ Min Cost for scalar multiplication = 20,000

6.

Longest Common Subsequence [LCS]

Given the DNA sequence

• $X = \text{"AGGTABGAC"}$

• $Y = \text{"GXTXAYBGZ"}$

Task: Find the longest common subsequence.

LCS → Sequence that appears in both the strings in the same order but not necessarily continuous.

Recurrence Relation

= = =

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1 & \text{if } x[i-1] = y[j-1] \\ \max(dp[i-1][j], dp[i][j-1]) & \text{if } x[i-1] \neq y[j-1] \end{cases}$$

(2)

if characters match, increase LCS length.

DP Table

	A	X	T	X	A	Y	B	G	Z
A	0	0	0	0	1	1	1	1	1
G	1	1	1	1	1	1	1	2	2
G	1	1	1	1	1	1	1	2	2
T	1	1	2	2	2	2	2	2	2
A	1	1	2	2	3	3	3	3	3
B	1	1	2	2	3	3	4	4	4
G	1	1	2	2	3	3	4	5	5
A	1	1	2	2	4	4	4	5	5
C	1	1	2	2	4	4	4	5	5

length of LCS = 5 ($dp[9][9] = 5$)

Common characters found : GTABG

LCS = "GTABG"

LCS length = 5